



# **Проектирование и разработка распределённого хранилища данных для семантической сети**

**Выполнил:** Барсуков М.А. Р3415

**Научный руководитель:** Цопа Е.А.

Университет ИТМО, 2026 г.

**Цель:** Обеспечение возможности горизонтального масштабирования хранения данных семантической сети.



**Задачи:**

- Проанализировать существующие подходы к распределенному хранению RDF-данных, выявить их архитектурные ограничения.
- Сформулировать требования к хранилищу и обосновать архитектуру.
- Спроектировать архитектуру кластера (модель хранения, стратегию шардирования, протокол репликации, алгоритм консенсуса узлов).
- Реализовать прототип с модулями хранения, кластеризацией и сетевым API.
- Обеспечить покрытие ключевых модулей тестами (целевой уровень — не менее 70%).
- Провести нагрузочное тестирование и тестирование отказоустойчивости.
- Сравнить полученные целевые характеристики с базовыми подходами (фиксированное и предикатное шардирование), оценить характеристики системы.



# Содержание работы

## Обоснование архитектуры:

- Анализ Neo4j, JanusGraph, Dgraph — выявлены ограничения в горизонтальном масштабировании, автономности и гибридных запросах
- Выбрана трёхуровневая архитектура: Data Plane + Control Plane + Gateway
- Data Plane — узлы с Raft-консенсусом и хранением поверх RocksDB.
- Control Plane — оркестратор для управления шардами и узлами.
- Gateway — единая точка входа с GraphQL.

## Технологии и инструменты:

- Rust (RocksDB, raft-rs, tokio, tonic, hnsw\_rs)
- Kotlin (Ktor, jetcd, graphql-kotlin)
- Python (pytest, E2E-тесты)
- Docker, etcd, Prometheus, Grafana

## Методы и алгоритмы:

- Семантическое шардирование на базе Leiden (обнаружение сообществ)
- Graph-Aware Compaction — учёт локальности графа при компактизации LSM-дерева
- HybridTime с commit-wait 2ε для External Consistency
- MVCC с HLC-таймштампами для Snapshot Isolation
- Bulk-load с одновременной записью в live и MVCC-столбцы
- Курсорная пагинация с привязкой к MVCC-снимку
- Predicate push-down для фильтрации на уровне шарда
- Cost-Based Optimizer (CBO) — выбор плана запроса на основе статистики
- BSP-движок для эффективных графовых обходов
- Автономные правила L1 и архитектура MAPE-K для самовосстановления

# Результаты экспериментальных исследований

## Производительность хранения:

- Bulk-load вершин — 2.5 млн/сек
- Bulk-load рёбер — 1.0 млн/сек
- BFS (fanout=4, depth=3) — 186 мкс
- KNN (10k×256, k=10) — 4.6 мс
- KNN (1k×64, k=100) — 188.0 μs (линейный поиск; HNSW — в процессе разработки)

## Пропускная способность

(по сценариям YCSB):

- Write-heavy (50/50) — 752 оп/сек, p50=9.65 мс, p95=19.4 мс, p99=28.4 мс
- Read-only (0/100) — 1010 оп/сек, p50=7.11 мс, p95=14.7 мс, p99=21.8 мс

## Отказоустойчивость:



- Выбор нового лидера при падении ноды — failover < 2 сек
- Восстановление репликации — автоматическое

## Масштабируемость:

- Семантическое шардирование — снижение межшардовых запросов на 60-80%
- В процессе разработки Multi-Raft для 1000+ нод

## Степень готовности:

- Разработан работающий прототип, подтвердивший основные гипотезы.
- Нагрузочное тестирование и тесты отказоустойчивости пройдены.
- План дальнейшей оптимизации и расширения функциональности составлен, в активной разработке.

**Спасибо  
за внимание!**

**it's**MO *re than a*  
**UNIVERSITY**